

13주차 강의

자바웹프로그래밍(1)

강사 : 최도현

전공 : S/W/네트워크/보안

주요 수업 과목(2011~ 현재)
프로그래밍/서버 OS 분야
C, C++, C#, JAVA, 자바웹(1, 2),
모바일, 운영체제, DB 등

새로운 시작!

오늘의 수업 내용

PART1 : 트렌드/이론, 지난주 복습

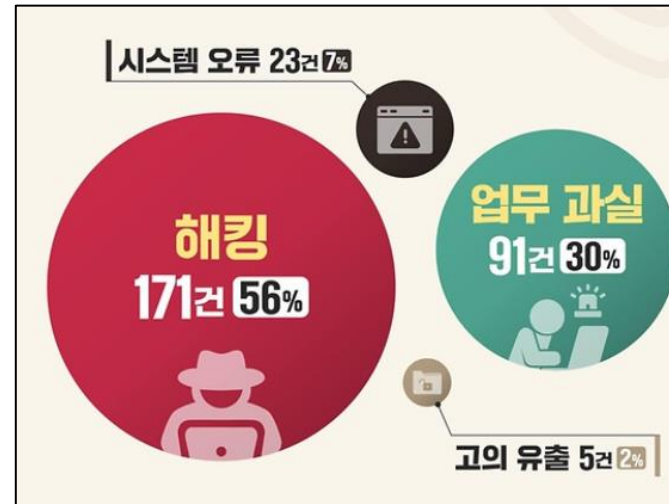
PART2 : 회원정보 수정 / 비밀번호 변경하기

PART3 : 마무리 & 과제



트렌드/이론 분석

- 2025 개인 정보 유출 현황
 - 대표 : 쿠팡, SK텔레콤
 - 전국민 정보...
- 유출 경로
 - 내부 : 내부자
 - 외부 : 해킹
- 주요 해킹 원인
 - 피싱 메일, 악성코드 등
- 보상안
 - 무슨 의미가 있을지...



쿠팡 개인정보 유출 주요 사례 및 일지

자료 : 쿠팡 · 개인정보보호위원회 · 사회민주당 한창민 의원

2020년 8월~'21년 11월	2021년 10월	2023년 12월	2025년 6월~11월
쿠팡이츠 배달원 개인정보 유출 배달원 실명, 휴대전화번호 등 음식점에 전달	앱 업데이트 간 테스트 소홀로 개인정보 유출	쿠팡 판매자 전용 시스템 '윙' 유출 주문자·수취인 개인정보 다른 판매자에게 노출	대규모 고객 정보 유출 발생 고객이름·이메일·전화번호·주소·일부 주문정보 노출
유출 규모 13만5000명	14건	2만2000명	3370만명

대규모 고객 정보 유출 사태 주요 일지

2025년 6월 24일

- 해의 서버를 통한 무단 개인정보 접근 시작 추정 (중국 국적의 쿠팡 전 직원 소행으로 추정, 퇴사 후 출국한 것으로 파악)

11월 18일

- 쿠팡, 유출 사실 최초 인지

11월 20일

- 쿠팡, 피해 사실 개인정보보호위원회에 신고 및 정보 유출 피해 고객 계정 4500여개로 발표

11월 25일

- 쿠팡, 경찰에 고소장 제출 (피고소인 '성명불상자'로 기재)

11월 29일

- 쿠팡, 추가 피해 사실 개인정보보호위원회에 신고 및 정보 유출 피해 고객 계정 3370만개로 정정해 공지

정부, 쿠팡 대규모 개인정보 유출사고 민관합동조사단 구성



SK텔레콤 해킹 사고 조사 결과

2025년 4월 18일 SKT 서버 침해 인지 이후 민관합동조사단 구성해
4월 23일~6월 27일 SKT 전체 서버 42,605대 조사 결과

피해 서버

서버 28대 감염 및 악성코드 33종 확인

※이번 사건과 무관한 것으로 보이는 악성코드 1종 유입 추가 확인

사고당시 SKT의 문제점

- 계정 정보 관리 부실: 음성통화인증(HSS) 관리 서버 계정 정보를 타 서버에 평문으로 저장
- 과거 침해사고 대응 미흡: SKT가 2022년 2월 23일 비정상적인 특이점을 발견하고도 별도 신고 조치 없이 자체적인 해결책으로 대응
- 중요 정보 암호화 조치 미흡: KT-LGU+ 등 타 통신사와 달리 유심 인증키를 암호화 없이 저장

유출 정보

전화번호, 가입자 식별번호(IMS) 등 유심정보 25종

(유출 규모 9.82GB, IMSI 기준 약 2,696만건. 가입자 전원의 유심 정보에 해당하는 분량)

※단말기 식별번호(IMEI) 및 개인정보가 평문으로 임시 저장된 서버가 감염, 최근 로그 기록 확인 결과 유출 정황은 없었지만 악성코드 감염 당시의 로그 기록이 없어 유출 여부를 정확히 알 수 없음

위약금 면제 적용 여부

SKT 이용약관 제43조

'회사의 귀책 사유'로 이용자가 서비스를 해지할 경우 위약금을 면제한다

→ 조사 결과 SKT의 과실이 있으며, SKT가 안전한 통신 서비스 제공이라는 계약의 주요 의무 위반을 했으므로 위약금 면제 규정에 해당

해킹 사고로 계약해지하는 이용자 대상

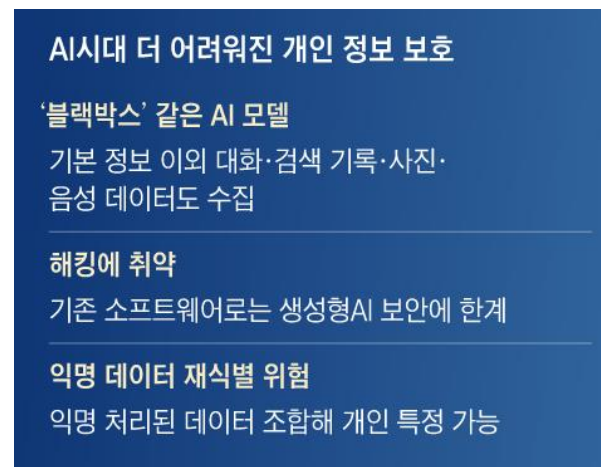
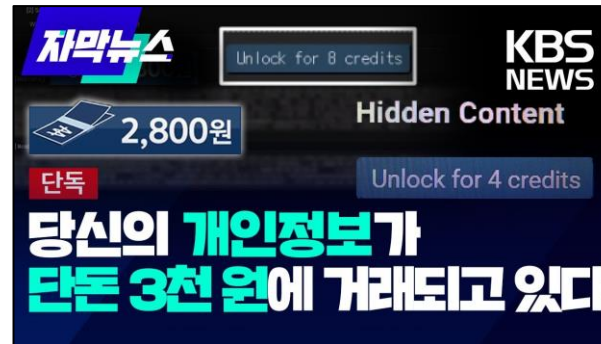
위약금 면제 필요

연방뉴스 자료: 과학기술정보통신부

김민지 기자 = minfo@yna.co.kr / 20250704

트렌드/이론 분석

- 중요 개인정보의 불법유통 확대
 - 개인을 식별 가능한 정보
 - 기본 식별, 연락처 등
 - 인적, 신체, 직업, 재산 등 가명 포함
- 중요 민감 정보는?
 - 주민번호, 의료정보, 금융정보
- 파생·자동 수집 정보는?
 - 나를 확인 가능(결합 가능성 원칙)
 - 다양한 장치에 식별 정보들 = 위협
- 최근 AI 모델 데이터 수집/생성
 - AI 챗봇, 딥페이크, 보이스피싱
 - 개인정보에 대한 노출/악용 등
- 프라이버시 보호 대안은?
 - 외부 전송 X, 로컬에서 부분적 최적화 등



구분	설명
일반적 정보	주민등록번호, 이름, 주소, 전화번호
통신 위치 정보	통화, IP주소, GPS 등
사회적 정보	교육 정보, 근로 정보, 자격 정보
정신적 정보	기호, 성향, 신념, 사상
신체적 정보	신체정보, 의료, 건강정보
재산적 정보	개인, 신용정보, 부동산, 주식



지난주 복습

- **지난주 내 홈페이지 확인**

- <http://localhost:8080/>
 - 프로필 페이지, 사진 업로드 테스트

- **오늘의 구현 목표**

- **프론트 수정**
 - Toast 알림으로 교체
 - 네비바 로그인 사용자명 표시
- **회원정보 수정**
 - 회원정보 수정 폼 / 엔드포인트
 - 비밀번호 변경 폼 / 엔드포인트
 - 비밀번호 변경 후 자동 로그아웃

 **내 프로필**



아이디	guest4
이메일	ddd@nate.xom
연락처	010-3333-3333

프로필 사진 변경

파일 선택

선택된 파일 없음

jpg, png, gif, webp / 최대 5MB

사진 업로드

도메인 패키지 구조

- 폴더 및 파일의 이름과 구조
 - 폴더 및 파일 구조
 - 기존 페이지의 알림을 모두 교체
 - 대부분 파일 유지
 - 기존 프로필 페이지를 수정
 - login 폴더 : profile.html 등
 - 참고 - DB 수정 x
 - 자바스크립트 수정 O
- 작업 완료 후 서버 재시작 X

오늘 추가/수정할 파일

```
org.acme/  
└── login/  
    └── AuthResource.java (수정: 각 엔드 포인트 추가)  
  
resources/META-INF/resources/  
├── main_index.html (수정: alert 제거)  
└── login/  
    ├── login.html (수정: alert 제거)  
    ├── register.html (수정: alert 제거)  
    ├── main_after_login.html (수정: 사용자명 표시)  
    └── profile.html (수정: 회원정보 수정 + 비번 변경 폼 추가)  
  
js/  
├── test.js (수정: alert → Toast)  
└── profile.js (수정: 수정 결과 오류 메시지 추가)
```

프론트 수정하기

- 브라우저 기본 알림 창
 - 부트스트랩5 기반 Toast로 구현
 - 특징 : 기존 작업과 병행 가능 - 비방해형
 - 참고 : 모달과는 다른 형태(자동 사라짐)
 - 현재 전체 HTML 페이지를 확인하자.
 - 스크립트를 찾아서 모두 제거/수정한다.

```
<script>
  window.onload = function() {
    alert("메인 페이지 로딩 완료"); ← 제거
    showToast('메인 페이지 로딩 완료 '); ← 추가
  }
</script>
```

- js 폴더의 test.js를 수정한다.

```
// test.js 수정 → Toast 함수 제공
```

```
function showToast(message, type = 'success') {
  // type : 'success' (초록) / 'danger' (빨강) / 'warning' (노랑)
  const toastEl = document.getElementById('liveToast');
  const toastBody = document.getElementById('toastBody');
```

```
  if (!toastEl || !toastBody) return;
```

```
  // 색상 클래스 변경
  toastEl.className =
    `toast align-items-center text-white bg-${type} border-0`;
  toastBody.textContent = message;
```

```
  // Bootstrap Toast 실행
  const toast = new bootstrap.Toast(toastEl, { delay: 3000 });
  toast.show();
}
```

구분	alert()	Toast
화면 차단	✓ 완전 차단	✗ 차단 없음
사용자 조작	버튼 클릭 필수	자동 사라짐
디자인	OS 기본 스타일	커스텀 가능
실제 서비스	✗ 거의 미사용	✓ 표준

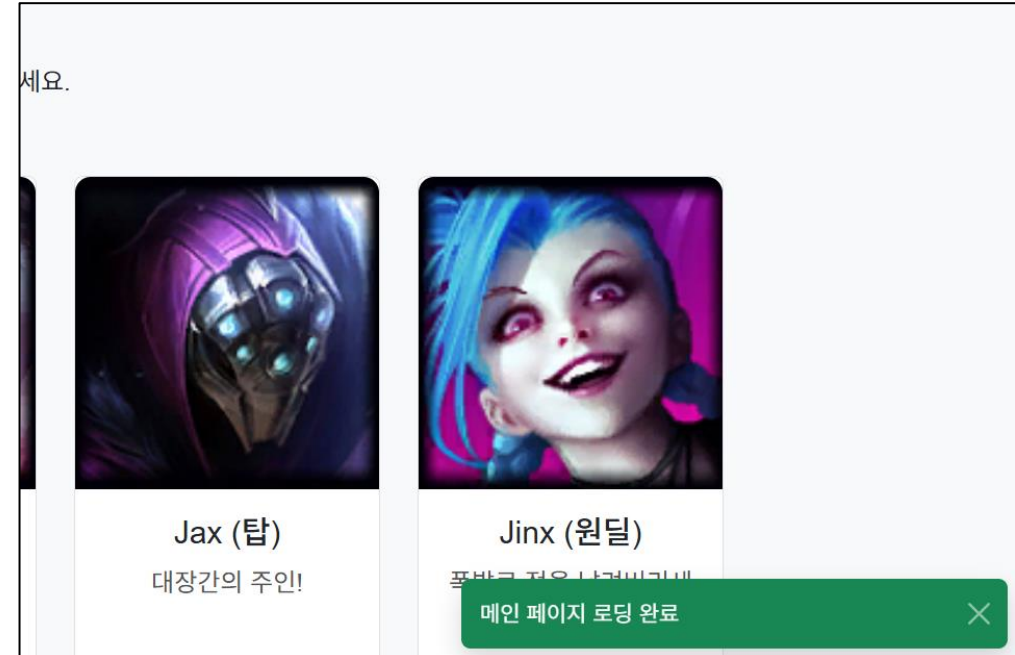
프론트 수정하기

- 브라우저 기본 알림 창
 - main_index.html에 Toast HTML 추가한다.

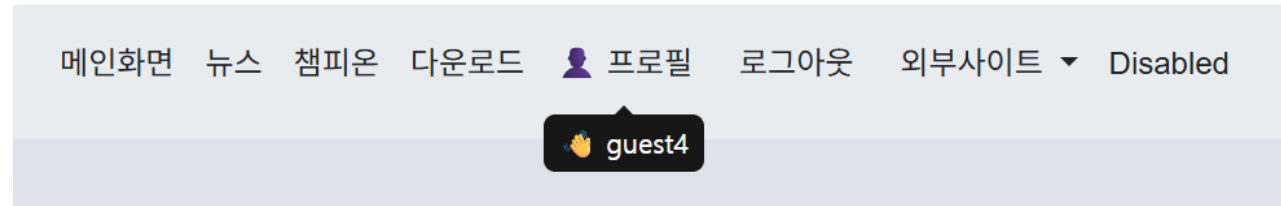
```
<!-- 토스트 컨테이너 </body> 바로 위에 추가 -->
<div class="toast-container position-fixed
    bottom-0 end-0 p-3" style="z-index:9999">
  <div id="liveToast" class="toast align-items-center
    text-white bg-success border-0" role="alert">
    <div class="d-flex">
      <div class="toast-body" id="toastBody">
        메시지
      </div>
      <button type="button"
        class="btn-close btn-close-white me-2 m-auto"
        data-bs-dismiss="toast"> </button>
    </div>
  </div>
</div>
```

- 저장 후 화면 확인

파일	메시지 예)
main_index.html	수정 완료
main_after_login.html	로그인 성공!
register_success.html	회원가입 완료!



프론트 수정하기



- 네비바의 사용자명 동적 표시
 - main_after_login.html 페이지와 js 폴더의 profile.js를 수정한다.

```
<!-- 기존 프로필 버튼 수정 -->
<a class="nav-link" href="/profile"
  id="profileNavLink"
  data-bs-toggle="tooltip"
  data-bs-placement="bottom">  프로필 </a>
```

```
// 회원 정보 읽고, json 형태 변환 후 화면 갱신(비동기 처리)
fetch('/profile/info')
  .then(res => res.json())
  .then(data => {
    const profileLink = document.getElementById('profileNavLink');
    if (profileLink) {
      profileLink.setAttribute('data-bs-title', '  ' + data.username);
      new bootstrap.Tooltip(profileLink);
    }
  });
```

코드	역할
data-bs-toggle="tooltip"	Bootstrap에게 이 요소가 tooltip임을 선언
data-bs-placement="bottom"	말풍선 방향 (top / bottom / left / right)
new bootstrap.Tooltip(el)	JS로 tooltip 초기화 (fetch 이후 동적 title이므로 필수)

회원정보 수정

- 회원정보 수정 폼 추가

- 기존 profile.htm을 수정한다.
 - Bootstrap의 Collapse(콜랩스), 상세 내용 숨기기/보기 기능

```
<!-- 사진 업로드 폼 아래에 추가 -->
<hr class="my-4">
<!-- 토글 버튼 -->
<button class="btn btn-warning w-100 mb-2"
        type="button"
        data-bs-toggle="collapse"
        data-bs-target="#updateFormArea"
        aria-expanded="false">
  📄 개인정보 수정
</button>
```

```
<!-- 기본 숨김 → 버튼 클릭 시 펼쳐짐 -->
<div class="collapse" id="updateFormArea">
  <div class="card card-body bg-dark
        text-white border-secondary mt-2">
    <!-- 수정 결과 메시지 -->
    <div id="updateMsg" class="alert d-none mb-3"> </div>
```

속성	값	설명
data-bs-toggle	collapse	클릭 시 접기/펼치기
data-bs-target	#updateFormArea	제어할 요소 id
class="collapse"	-	기본 숨김 상태
class="collapse show"	-	기본 펼친 상태

```
<form id="updateForm"
      method="POST"
      action="/profile/update">
```

```
<div class="mb-3 text-start">
  <label class="form-label">이메일 </label>
  <input type="text" class="form-control"
        id="updateEmail" name="email"
        placeholder="example@email.com">
  <div class="invalid-feedback" id="updateEmailMsg"> </div>
</div>
```

회원정보 수정

- 회원정보 수정 폼 추가
 - 기존 profile.htm을 수정한다. 저장 후 화면 확인
 - 참고 : js 없이도 화면 숨김이 가능하다.

```
<div class="mb-3 text-start">  
  <label class="form-label">연락처</label>  
  <input type="text" class="form-control"  
    id="updatePhone" name="phone"  
    placeholder="010-0000-0000">  
  <div class="invalid-feedback" id="updatePhoneMsg"></div>  
</div>
```

```
<button type="button"  
  class="btn btn-warning w-100"  
  onclick="validateAndUpdate()">  
  수정 완료  
</button>  
</form>  
</div>  
</div>
```

개인정보 수정

이메일

example@email.com

연락처

010-0000-0000

수정 완료

회원정보 수정

- 회원정보 수정 폼 - 기존 값 자동 채움
 - 기존 profile.js를 수정한다. Fetch(db 정보 표시)
 - 저장 후 화면 새로고침

```
fetch('/profile/info')
  .then(res => res.json())
  .then(data => {
    // 기존 정보 테이블 표시 (유지)
    document.getElementById('infoUsername').textContent = data.username;
    document.getElementById('infoEmail').textContent = data.email;
    document.getElementById('infoPhone').textContent = data.phone;
    if (data.profileImage) {
      document.getElementById('profileImg').src =
        '/uploads/profile/' + data.profileImage;
    }
  })
```

```
// ✅ 수정 폼에 기존 값 자동 채우기
document.getElementById('updateEmail').value = data.email;
document.getElementById('updatePhone').value = data.phone;
```

```
// ✅ Tooltip 으로 사용자명 표시 (navUsername span 방식 → 교체)
const profileLink = document.getElementById('profileNavLink');
if (profileLink) {
  profileLink.setAttribute('data-bs-title', '👋 ' + data.username);
  new bootstrap.Tooltip(profileLink);
}
});
```

코드	설명
data-bs-toggle="tooltip"	마우스 오버 시 말풍선 표시
data-bs-placement="bottom"	툴팁 표시 방향 (top/bottom/left/right)
data-bs-title	툴팁에 표시할 텍스트
setAttribute('data-bs-title', ...)	JS로 동적으로 속성값 변경

개인정보 수정

이메일

ddd@nate.xom

연락처

010-3333-3333

수정 완료

회원정보 수정

- 회원정보 수정 폼 - 정규식 검사
 - 기존 profile.js를 수정한다.
 - 참고 : 기존 정규식 검사와 같이 동작

```
function validateAndUpdate() {  
    let valid = true;
```

```
    const email = document.getElementById('updateEmail').value.trim();  
    const phone = document.getElementById('updatePhone').value.trim();
```

```
    // ① 이메일 형식 검사  
    const emailRegex = /^[^Ws@]+@[^Ws@]+\w.[^Ws@]+$/;  
    if (!emailRegex.test(email)) {  
        showFieldError('updateEmail', 'updateEmailMsg',  
            '올바른 이메일 형식이 아닙니다.');
```

```
        valid = false;
```

```
    } else {  
        clearFieldError('updateEmail');
```

```
    }
```

```
    // ② 연락처 형식 검사  
    const phoneRegex = /^010-\w{4}-\w{4}$/;  
    if (!phoneRegex.test(phone)) {  
        showFieldError('updatePhone', 'updatePhoneMsg',  
            '010-0000-0000 형식으로 입력해주세요.');
```

```
        valid = false;
```

```
    } else {  
        clearFieldError('updatePhone');
```

```
    }
```

```
    if (valid) document.getElementById('updateForm').submit();  
}
```

```
// profile.js 전용 showError / clearError  
function showFieldError(fieldId, msgId, message) {  
    const field = document.getElementById(fieldId);  
    field.classList.add('is-invalid');
```

```
    const msg = document.getElementById(msgId);  
    if (msg) msg.textContent = message;
```

```
function clearFieldError(fieldId) {  
    const field = document.getElementById(fieldId);  
    field.classList.remove('is-invalid');
```

```
    field.classList.add('is-valid');
```

```
}
```

회원정보 수정

- 회원정보 수정 - 엔드포인트
 - 기존 AuthResource.java를 수정한다.
 - 참고 : dev ui에서 엔드포인트 확인

```
@POST
@Path("/profile/update")
@Transactional
@Consumes(MediaType.APPLICATION_FORM_URLENCODED)
public Response profileUpdate(
    @FormParam("email") String email,
    @FormParam("phone") String phone) {
```

```
// ① 세션 체크
String loginUser = context.session().get("loginUser");
if (loginUser == null) {
    return Response
        .seeOther(URI.create("/login"))
        .build();
}
```

처리	결과
세션 없음	/login 차단
이메일 중복	/profile?error=duplicate_email
정상 수정	DB 업데이트 → /profile?success=updated

```
// ② 이메일 중복 체크 (본인 제외)
User found = User.findByEmail(email);
if (found != null && !found.username.equals(loginUser)) {
    return Response
        .seeOther(URI.create("/profile?error=duplicate_email"))
        .build();
}
```

```
// ③ DB 업데이트
User user = User.findByUsername(loginUser);
user.email = email;
user.phone = phone;
```

```
return Response
    .seeOther(URI.create("/profile?success=updated"))
    .build();
}
```

회원정보 수정

- 회원정보 수정 – 결과 및 메시지
 - 기존 profile.js를 수정한다. 수정 후 클릭

```
window.onload = function() {  
  // 기존 fetch 코드 전체 유지
```

```
  // URL 파라미터 오류 감지  
  const params = new URLSearchParams(window.location.search);  
  const error = params.get('error');  
  const success = params.get('success');
```

```
  const msgEl = document.getElementById('updateMsg');
```

```
  if (success === 'updated') {  
    msgEl.className = 'alert alert-success';  
    msgEl.textContent = '✅ 개인정보가 수정되었습니다.';  
  } else if (error === 'duplicate_email') {  
    msgEl.className = 'alert alert-danger';  
    msgEl.textContent = '⚠️ 이미 사용 중인 이메일입니다.';  
  }  
}
```

```
if (error === 'wrong_password') {  
  // ① Toast 먼저 (즉각 알림)  
  showToast('⚠️ 현재 비밀번호가 일치하지 않습니다.', 'danger');  
  const pwMsgEl = document.getElementById('pwMsg');  
  if (pwMsgEl) {  
    pwMsgEl.className = 'alert alert-danger';  
    pwMsgEl.textContent = '⚠️ 현재 비밀번호가 일치하지 않습니다.';  
  }  
}
```

```
if (error) {  
  const messages = {  
    'invalid_type': 'jpg, png, gif, webp 파일만 가능합니다.',  
    'too_large': '파일 크기는 5MB 이하여야 합니다.',  
    'upload_fail': '업로드 실패. 다시 시도해주세요.'  
  };  
  const msg = messages[error];  
  const div = document.getElementById('uploadErrorMsg');  
  if (msg && div) {  
    div.textContent = msg;  
    div.classList.remove('d-none');  
  }  
}
```


비밀번호 변경하기

- 비밀번호 변경 폼 추가
 - profile.html을 수정한다.

```
<!-- 개인정보 수정 폼 아래에 추가 -->
```

```
<hr class="my-4">
```

```
<h5 class="fw-bold mb-3">🔒 비밀번호 변경</h5>
```

```
<!-- 결과 메시지 -->
```

```
<div id="pwMsg" class="alert d-none mb-3"></div>
```

```
<form id="pwForm"
```

```
  method="POST"
```

```
  action="/profile/password">
```

```
<div class="mb-3 text-start">
```

```
  <label class="form-label">현재 비밀번호</label>
```

```
  <!-- 입력용 (서버 전송 안 됨) -->
```

```
  <input type="password" class="form-control"
```

```
    id="currentPwInput"
```

```
    placeholder="현재 비밀번호 입력" required>
```

input	id	name	서버 전송
현재 PW 입력	currentPwInput	없음	✗
현재 PW hidden	currentPassword	currentPassword	✓
새 PW 입력	newPwInput	없음	✗
새 PW hidden	newPassword	newPassword	✓
새 PW 확인	newPwConfirm	없음	✗

```
<div class="invalid-feedback" id="currentPwMsg"></div>
```

```
  <!-- 해시값 전송용 hidden -->
```

```
  <input type="hidden" id="currentPassword"
```

```
    name="currentPassword">
```

```
</div>
```

```
<div class="mb-3 text-start">
```

```
  <label class="form-label">새 비밀번호</label>
```

```
  <input type="password" class="form-control"
```

```
    id="newPwInput"
```

```
    placeholder="8자 이상, 영문+숫자+특수문자" required>
```

```
  <div class="invalid-feedback" id="newPwMsg"></div>
```

```
  <input type="hidden" id="newPassword"
```

```
    name="newPassword">
```

```
</div>
```

비밀번호 변경하기

- 비밀번호 변경 폼 추가
 - profile.html을 수정한다. (자동 펼침 x)

```
<div class="mb-3 text-start">  
  <label class="form-label">새 비밀번호 확인 </label>  
  <input type="password" class="form-control"  
    id="newPwConfirm"  
    placeholder="새 비밀번호 재입력" required>  
  <div class="invalid-feedback" id="newPwConfirmMsg"> </div>  
</div>
```

```
<button type="button"  
  class="btn btn-danger w-100"  
  onclick="validateAndChangePassword()">  
  비밀번호 변경  
</button>  
</form>
```

- 참고 : toast 기능을 위해 test.js 연동 추가

 **비밀번호 변경**

현재 비밀번호

현재 비밀번호 입력

새 비밀번호

8자 이상, 영문+숫자+특수문자

새 비밀번호 확인

새 비밀번호 재입력

비밀번호 변경

비밀번호 변경하기

- 비밀번호 유효성 검사 + 해시
 - profile.js를 수정한다.

```
async function validateAndChangePassword() {  
  let valid = true;
```

```
  const currentPw = document.getElementById('currentPwInput').value;  
  const newPw = document.getElementById('newPwInput').value;  
  const newPwConfirm = document.getElementById('newPwConfirm').value;
```

```
  // ① 현재 비밀번호 빈 값 체크
```

```
  if (!currentPw) {  
    showFieldError('currentPwInput', 'currentPwMsg',  
      '현재 비밀번호를 입력해주세요.');
```

```
    valid = false;  
  } else {  
    clearFieldError('currentPwInput');
```

```
  }
```

검사 항목	조건
현재 비밀번호	빈 값 방지
새 비밀번호	8자+영문+숫자+특수문자
새 비밀번호 확인	새 비밀번호와 일치
해시 처리	현재·새 비밀번호 모두 SHA-256

```
  // ② 새 비밀번호 정규식 검사
```

```
  const pwRegex = /^(?=.*[a-zA-Z])(?=.*\d)(?=.*[!@#$%^&*]).{8,}$/;  
  if (!pwRegex.test(newPw)) {  
    showFieldError('newPwInput', 'newPwMsg',  
      '8자 이상, 영문+숫자+특수문자를 포함해야 합니다.');
```

```
    valid = false;  
  } else {  
    clearFieldError('newPwInput');
```

```
  }
```

```
  // ③ 새 비밀번호 확인 일치
```

```
  if (newPw !== newPwConfirm) {  
    showFieldError('newPwConfirm', 'newPwConfirmMsg',  
      '새 비밀번호가 일치하지 않습니다.');
```

```
    valid = false;  
  } else {  
    clearFieldError('newPwConfirm');
```

```
  }
```

비밀번호 변경하기

- 비밀번호 유효성 검사 + 해시
 - profile.js를 수정한다.

```
if (!valid) return;
```

```
// ④ 현재/새 비밀번호 SHA-256 해시 생성  
const hashedCurrent = await hashPassword(currentPw);  
const hashedNew      = await hashPassword(newPw);
```

```
document.getElementById('currentPassword').value = hashedCurrent;  
document.getElementById('newPassword').value     = hashedNew;
```

```
// F12 콘솔 확인  
console.log('현재 PW 해시 :', hashedCurrent);  
console.log('새 PW 해시   :', hashedNew);
```

```
document.getElementById('pwForm').submit();  
}
```

- 저장 후 확인(정규식만 검사됨)

 비밀번호 변경

현재 비밀번호



새 비밀번호



8자 이상, 영문+숫자+특수문자를 포함해야 합니다.

새 비밀번호 확인



새 비밀번호가 일치하지 않습니다.

비밀번호 변경

비밀번호 변경하기

- 비밀번호 변경 – 엔드포인트
 - AuthResource.java를 수정한다.

```
@POST
@Path("/profile/password")
@Transactional
@Consumes(MediaType.APPLICATION_FORM_URLENCODED)
public Response profilePassword(
    @FormParam("currentPassword") String currentPassword,
    @FormParam("newPassword") String newPassword) {
```

```
// ① 세션 체크
String loginUser = context.session().get("loginUser");
if (loginUser == null) {
    return Response
        .seeOther(Uri.create("/login"))
        .build();
}
```

처리	결과
세션 없음	/login 차단
현재 PW 불일치	/profile?error=wrong_password
변경 성공	DB 업데이트 → 변경 성공

```
// ② 현재 비밀번호 확인 (해시값 비교)
User user = User.findByUsername(loginUser);
if (!user.password.equals(currentPassword)) {
    return Response
        .seeOther(Uri.create("/profile?error=wrong_password"))
        .build();
}
```

```
// ③ 새 비밀번호로 DB 업데이트
user.password = newPassword;
```

```
return Response
    .seeOther(Uri.create("/profile?success=password_changed"))
    .build();
}
```

비밀번호 변경하기

- 비밀번호 변경 – 엔드포인트(로그 아웃)
 - AuthResource.java를 수정한다.
 - 참고 : 비밀번호 변경 후 로그인 페이지로 전환

```
@GET
@Path("/logout") // 기존에는 로그아웃 하면 항상 메인 이동
public Response logout(@QueryParam("next") String next) { // ← next 파라미터 추가
    context.session().destroy();
    String redirect = (next != null && next.equals("login")) ? "/login" : "/";
    return Response
        .seeOther(URI.create(redirect)) // ← ?next=login 이면 /login으로
        .build();
}
```

호출 URL	next 값	이동 경로
/logout	null	/ (메인)
/logout?next=login	"login"	/login
/logout?next=other	"other"	/ (메인)

항목	기존	변경	설명
파라미터	없음	@QueryParam("next") String next	URL의 ?next= 값을 String next 변수로 자동 추출
이동 경로	/ 고정	next 값에 따라 분기	상황에 따라 다른 페이지로 이동
@QueryParam	미사용	신규 사용	URL ?next=login 값 추출
삼항 연산자	없음	? "/login" : "/"	조건에 따라 경로 결정

비밀번호 변경하기

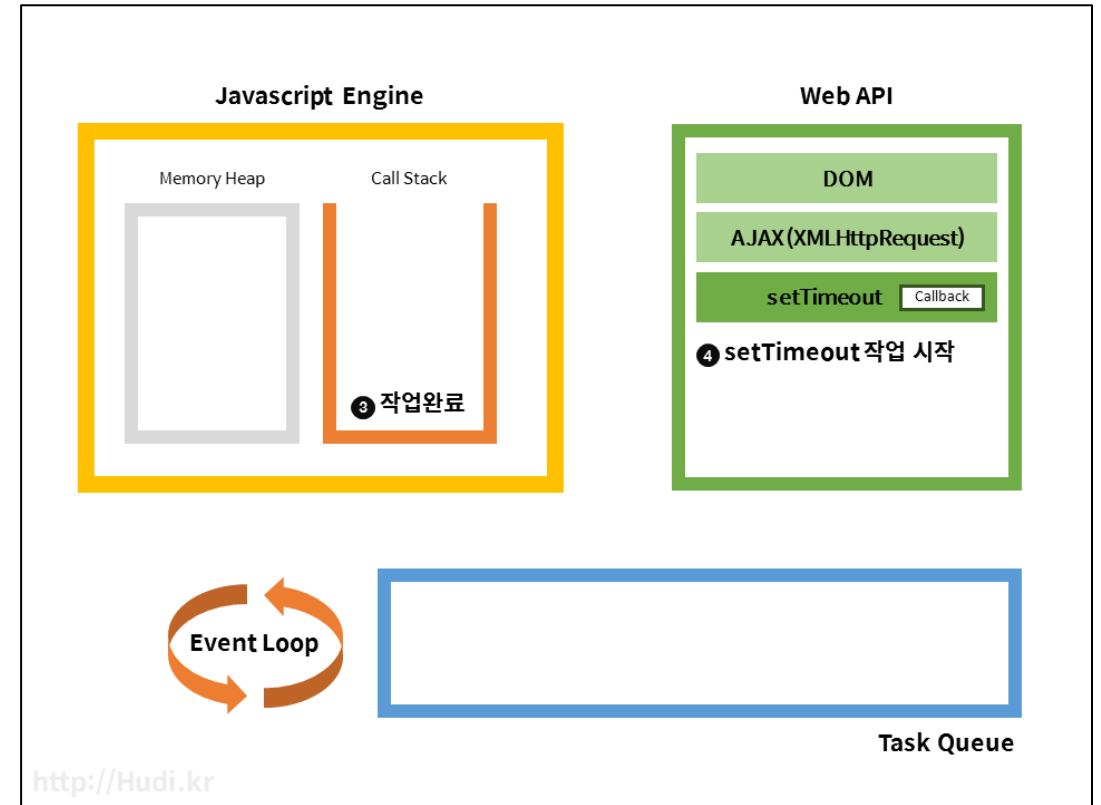
- 비밀번호 변경 – 성공 Toast 처리
 - profile.js를 수정한다. Js에서 토스트 호출

```
// 비밀번호 변경 성공 처리, window.onload 안에 삽입  
if (success === 'password_changed') {
```

```
    // Toast 출력  
    showToast(  
        '✅ 비밀번호가 변경 완료, 로그인 페이지로 이동합니다.',  
        'success'  
    );
```

```
    // 3.5초 후 로그인 페이지로 이동  
    setTimeout(function() {  
        window.location.href = '/logout?next=login';  
    }, 3500);  
}
```

- 참고 : fetch 밖에 처리해야 함



항목	설명
역할	지정한 시간(ms) 이 지난 후 함수를 실행
형식	setTimeout(실행할함수, 대기시간ms)
단위	ms (밀리초) / 1000ms = 1초
3500	3500ms = 3.5초 대기 후 실행
비동기	대기 중에도 다른 코드 실행 가능 (블로킹 없음)
취소 방법	clearTimeout(id) 로 취소 가능

비밀번호 변경하기

- 비밀번호 변경 – Toast 처리 추가
 - Profile.html를 수정한다.
 - 참고 : 암호화에 input_sha256 연동 필요

```
<!-- Toast 컨테이너 추가 -->  
<div class="toast-container position-fixed bottom-0 end-0 p-3" style="z-index:9999">  
  <div id="liveToast" class="toast align-items-center text-white bg-success border-0" role="alert">  
    <div class="d-flex">  
      <div class="toast-body" id="toastBody">메시지</div>  
      <button type="button" class="btn-close btn-close-white me-2 m-auto" data-bs-dismiss="toast"></button>  
    </div>  
  </div>  
</div>
```

- 저장 후 비밀번호 변경 수행

✓ 비밀번호가 변경되었습니다. 잠시 후 로그인 페이지로 이동합니다.

마무리 & 과제

- **깃 허브 업로드**

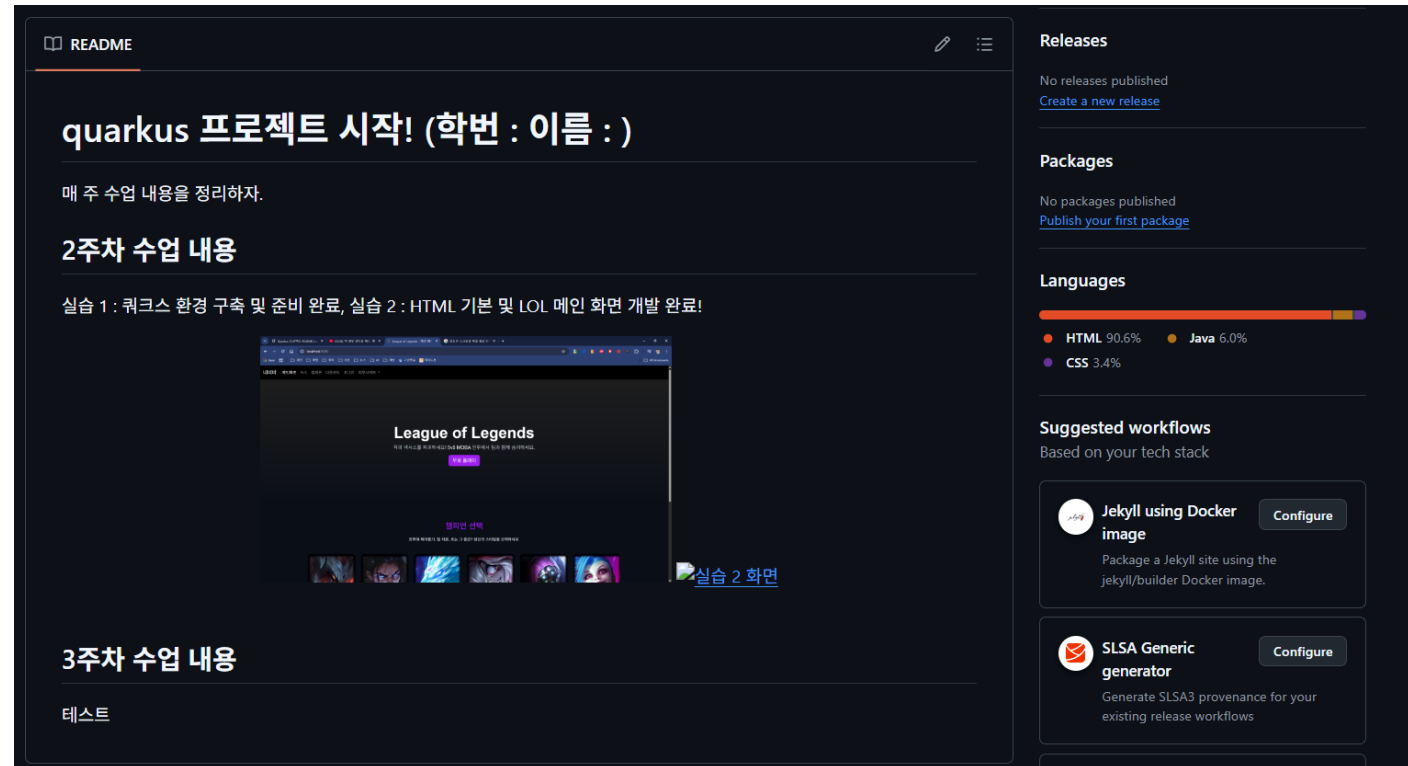
- VS CODE 메시지 입력
- "x주차 수업 내용 업로드"
- 입력 후 커밋 버튼 클릭

- **readme.md 파일 업데이트**

- 수업 주차 내용 요약
- 깃 페이지의 메인화면

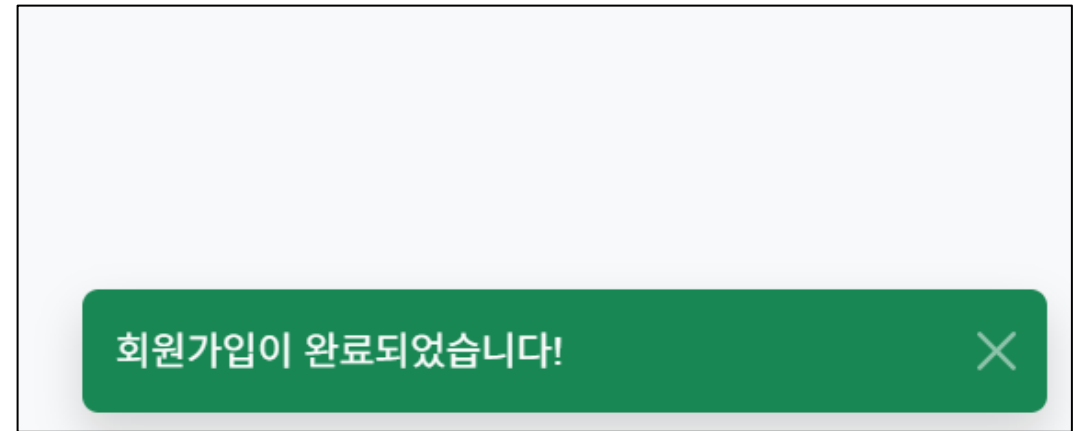
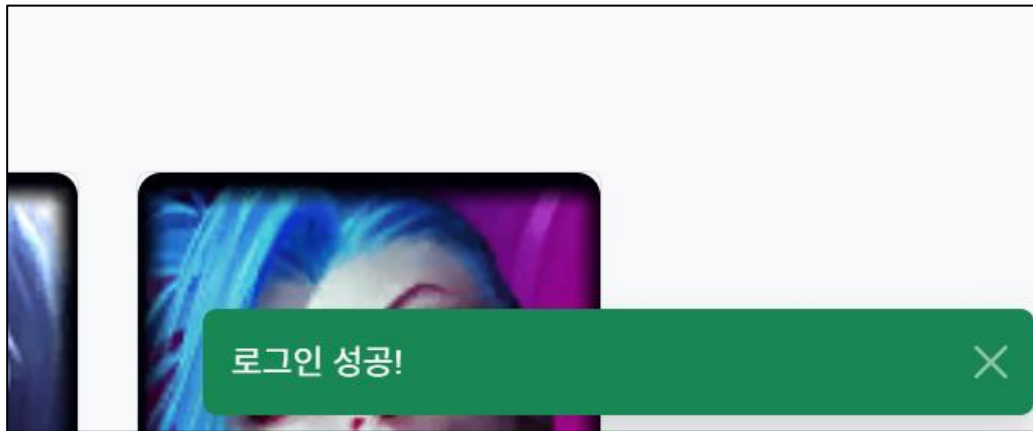
- **깃 허브 웹 사이트**

- <https://github.com/>
- 내 저장소 메인 화면 확인



마무리 & 과제

- 토스트로 교체하기(슬라이드 15 참고)
 - 기존 미구현된 alert 들을 Toast로 구현(js 스크립트 내부 호출 형태로)



파일	window.onload 처리	onclick alert
main_index.html	✓ showToast('메인 페이지 로딩 완료')	✓ showToast('즐거운 플레이 되세요')
login/main_after_login.html	✗ alert 삭제만 됨 — toast 없음	✓ showToast('즐거운 플레이 되세요')
login/register.html	✗ alert 삭제만 됨 — toast 없음	✓ '회원가입 페이지 로딩 완료'
login/register_success.html	✗ alert 삭제만 됨 — toast 없음	✓ '회원가입이 완료되었습니다!'

마무리 & 과제

- 최종 마무리 작업
 - 메인페이지
 - 모든 페이지에서 검색 창 동작 확인
 - Js 로드 순서 확인, 상대경로 문제 등
 - 네비게이션 바 통일 및 링크 완성
 - 모든 하이퍼 링크를 확인 : 링크 없음, 상대경로 오류 등 체크
 - 외부 사이트 연결과 Disabled 불필요한 항목 제거
 - 전체 페이지에서 다크/라이트 모드 통일
 - 세션을 활용하면 다크/라이트 모드 유지 가능
 - 자바코드, 자바스크립트 코드
 - 들여쓰기, 불필요한 코드 중복 체크
 - 코드 별 주석처리



Q & A

- 무엇이든 물어보세요!
 - 수업 내용, 과제, 개발 환경, 진로 상담 등

